BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC SÀI GÒN

Môn: Quản Trị Mạng

Đề tài: Triển khai hệ thống Hosting

E-commerce Đa nền tảng

Thành viên nhóm
3124
3124
3124
3124
3124

Giảng viên hướng dẫn: Thầy Lương Minh Huấn

MỤC LỤC

LỜI NÓI ĐẦU 3
CHƯƠNG 1. PHÂN TÍCH ĐỀ TÀI 4
1.1. Tổng quan và lý do chọn đề tài 4
1.2. Mục tiêu của đề tài 4
1.3. Phạm vi và yêu cầu hệ thống 4
1.4. Phân tích mô hình triển khai 5
1.5. Sơ đồ kiến trúc mạng 6
1.6. Sơ đồ luồng hoạt động hệ thống 7
1.7. Phân tích các chức năng chính 7
CHƯƠNG 2. TRIỂN KHAI KỸ THUẬT 9
2.1. Môi trường triển khai 9
2.2. Thiết lập Tailscale VPN Mesh 9
2.3. Cấu hình VM1 – NGINX Reverse Proxy và Load Balancer 10
2.4. Cấu hình VM2 – Linux Apache Web Server 14
2.5. Cấu hình VM3 – Windows IIS Web Server 14
2.6. Cấu hình VM4 – Database & Object Storage Server 15
2.7. Cấu hình VM5 – FTP Server 12
2.8. Cấu hình phân giải tên miền nội bộ 16
2.9. Tích hợp hệ thống và quy trình kiểm thử 16
CHƯƠNG 3. KẾT QUẢ VẬN HÀNH 17
3.1. Kết quả triển khai tổng thể 17
3.2. Kiểm thử truy cập website 17
3.3. Kiểm thử cân bằng tải 17
3.4. Kiểm thử khả năng chịu lỗi 18
3.5. Kiểm thử cơ sở dữ liệu và FTP 18
3.6. Kết quả giám sát và kiểm thử tải 20
3.7. Đánh giá kết quả và các vấn đề còn tồn tại 20
CHƯƠNG 4. KẾT LUẬN 22
4.1. Kết quả đạt được 22
4.2. Hạn chế 22
4.3. Hướng phát triển 22
TÀI LIỆU THAM KHẢO 23
PHỤ LỤC 24
Phụ lục A. Cấu hình NGINX đầy đủ 24
Phụ lục B. Lệnh kiểm thử 25
Phụ lục C. Lưu ý khi nộp báo cáo 26

LỜI NÓI ĐẦU

Trong bối cảnh công nghệ thông tin phát triển mạnh mẽ, các hệ thống thương mại điện tử ngày càng đòi hỏi cao về khả năng truy cập, tính ổn định và hiệu quả quản lý hạ tầng máy chủ. Việc triển khai một website thương mại điện tử không chỉ là xây dựng ứng dụng mà còn cần tổ chức hệ thống máy chủ, kết nối mạng, lưu trữ dữ liệu và giám sát hoạt động một cách hợp lý.

Xuất phát từ yêu cầu đó, nhóm lựa chọn đề tài "Triển khai hệ thống Hosting E-commerce Đa nền tảng", sử dụng ứng dụng Phone Store làm hệ thống minh họa. Đề tài xây dựng mô hình hosting gồm các máy chủ ảo chạy trên các hệ điều hành khác nhau, kết nối với nhau thông qua mạng chung. Trong mô hình, NGINX đảm nhiệm vai trò reverse proxy và cân bằng tải; Apache (Linux) và IIS (Windows) cung cấp dịch vụ web; máy chủ cơ sở dữ liệu lưu trữ dữ liệu tập trung; máy chủ FTP hỗ trợ quản lý và triển khai mã nguồn.

Thông qua quá trình thực hiện, nhóm vận dụng các kiến thức về quản trị mạng: cấu hình dịch vụ máy chủ, kết nối cơ sở dữ liệu từ xa, cân bằng tải, xử lý sự cố và giám sát hệ thống. Báo cáo trình bày quá trình phân tích, thiết kế, triển khai kỹ thuật và kiểm thử mô hình, từ đó đánh giá kết quả đạt được, hạn chế còn tồn tại và hướng phát triển.

[Ghi chú: chỉ giữ lại những nội dung đúng với những gì nhóm thực sự đã thực hiện. Xóa mọi ghi chú tô vàng trước khi nộp.]

CHƯƠNG 1. PHÂN TÍCH ĐỀ TÀI

1.1. Tổng quan và lý do chọn đề tài

Khi triển khai hosting cho một website thương mại điện tử, một số vấn đề cần giải quyết gồm:

- Nếu chỉ dùng một web server, toàn bộ hệ thống phụ thuộc vào máy chủ đó; khi máy chủ gặp sự cố, website ngừng phục vụ.
- Cần phân phối truy cập giữa nhiều máy chủ để minh họa cơ chế cân bằng tải.
- Cần tổ chức lưu trữ dữ liệu dùng chung để các web server cùng truy cập một nguồn dữ liệu.
- Cần phương thức quản lý và triển khai mã nguồn từ xa một cách an toàn.
- Cần theo dõi lưu lượng truy cập và kiểm thử khả năng phục vụ của hệ thống.

Từ các vấn đề trên, nhóm lựa chọn giải pháp là mô hình Phone Store chạy trên cụm máy ảo kết nối qua Tailscale VPN Mesh, trong đó mỗi máy chủ đảm nhiệm một vai trò riêng.

1.2. Mục tiêu của đề tài

Mục tiêu chung: xây dựng và kiểm thử mô hình hosting thương mại điện tử đa nền tảng, trong đó các dịch vụ web, cơ sở dữ liệu và quản lý mã nguồn được phân tách trên nhiều máy chủ.

Mục tiêu cụ thể:

1. Thiết lập mạng riêng kết nối các máy ảo thông qua Tailscale.
2. Cấu hình NGINX làm reverse proxy và cân bằng tải giữa Apache và IIS.
3. Triển khai cơ sở dữ liệu tập trung để các web server kết nối từ xa.
4. Thiết lập FTP và cơ chế cô lập người dùng (chroot) để hỗ trợ triển khai mã nguồn.
5. Kiểm thử cân bằng tải, khả năng chuyển đổi khi máy chủ gặp sự cố và hiệu năng thông qua các công cụ giám sát.

1.3. Phạm vi và yêu cầu hệ thống

Nhóm yêu cầu	Nội dung
Yêu cầu chức năng	Truy cập website Phone Store, sử dụng chức năng quản trị, truy xuất dữ liệu qua RESTful API, triển khai mã nguồn và giám sát truy cập.
Yêu cầu hạ tầng	Có nhiều máy chủ web, một máy chủ cơ sở dữ liệu, máy chủ FTP và mạng kết nối giữa các máy.

Nhóm yêu cầu	Nội dung
Yêu cầu vận hành	Cân bằng tải, kiểm thử chuyển đổi khi một web server gặp sự cố, kiểm tra kết nối dữ liệu từ xa và kiểm thử tải.
Phạm vi kiểm thử	Chỉ xác nhận những chức năng đã cấu hình và kiểm thử thành công trong môi trường thực tế của nhóm.

Do đây là đồ án môn Quản trị mạng, báo cáo tập trung vào hạ tầng hosting (mạng, máy chủ, cân bằng tải, cơ sở dữ liệu, FTP, giám sát). Ứng dụng Phone Store chỉ đóng vai trò hệ thống minh họa để kiểm chứng hạ tầng.

1.4. Phân tích mô hình triển khai

Hệ thống gồm 5 máy ảo, kết nối qua mạng phủ Tailscale VPN Mesh (dải địa chỉ 100.64.0.0/10). Vai trò và địa chỉ của từng máy được tổng hợp ở Bảng 1.1.

Bảng 1.1. Quy hoạch máy chủ và địa chỉ Tailscale

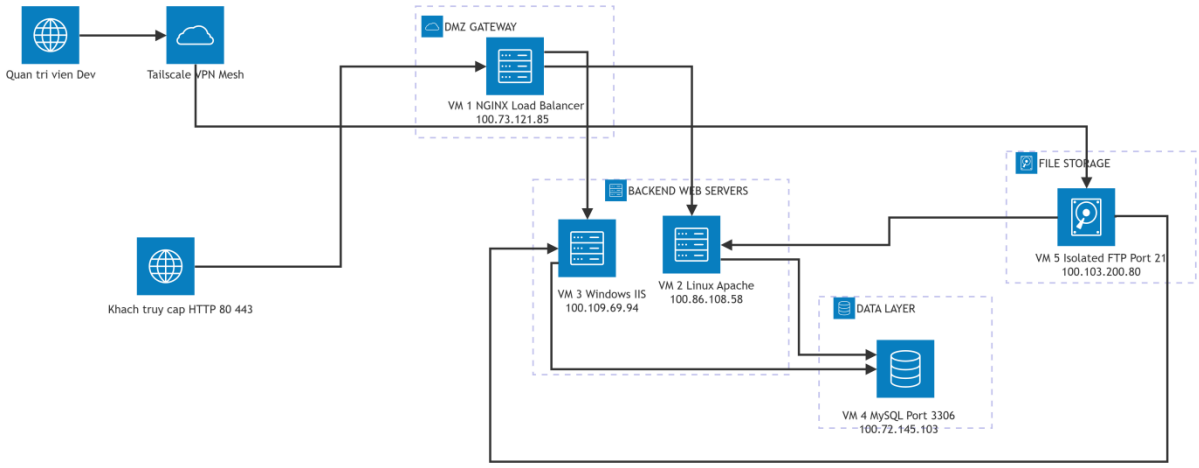
Máy ảo	Hệ điều hành	Dịch vụ / Port	Địa chỉ Tailscale	Vai trò
VM1	AlmaLinux	NGINX (80, 443), GoAccess (7890)	`100.73.121.85`	Reverse Proxy & Load Balancer
VM2	CentOS	Apache + PHP (80)	`100.86.108.58`	Backend Web Server 1
VM3	Windows Server	IIS + PHP (80)	`100.109.69.94`	Backend Web Server 2
VM4	Linux (Ubuntu)	MariaDB/MySQL (3306) , MinIO (9000, 9001)	`100.72.145.103`	Database Server & Object Storage (MinIO)
VM5	Linux [xác nhận]	vsftpd (21)	`100.103.200.80`	FTP Server cô lập người dùng

[Các địa chỉ trên lấy từ tài liệu dự án; hãy kiểm tra lại IP, hệ điều hành và dịch vụ thực tế trên từng máy trước khi nộp.]

Về ứng dụng, Phone Store gồm giao diện khách hàng viết bằng React (SPA), trang quản trị (Admin) và cụm RESTful API viết bằng PHP, kết nối MySQL qua PDO. Mã nguồn dự kiến được triển khai giống nhau trên cả Apache và IIS; mỗi node có một banner nhận diện (file config.js) để phục vụ việc kiểm tra cân bằng tải. Tuy nhiên, trong quá trình vận hành thực tế, hai node đang chạy hai phiên bản giao diện khác nhau (VM2 – CentOS chạy bản "Techno Store", VM3 – Windows chạy bản "Phone Store") do chưa có cơ chế

đồng bộ mã nguồn tự động, phải sao chép thủ công lên từng máy (xem hạn chế ở mục 4.2).

1.5. Sơ đồ kiến trúc mạng

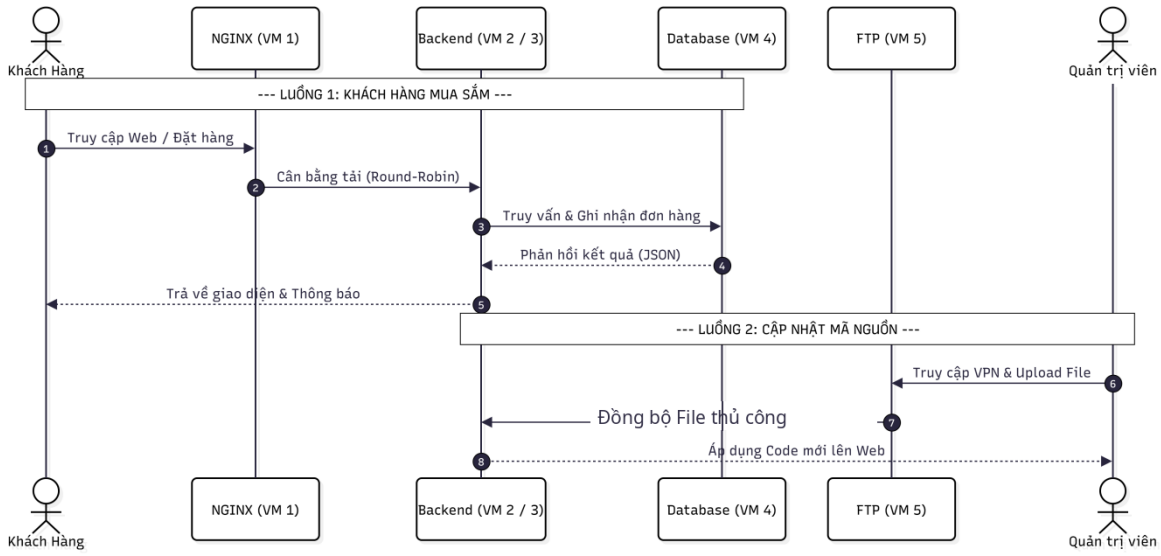


Hình 1.1. Sơ đồ kiến trúc mạng của hệ thống Hosting E-commerce đa nền tảng

Các ý chính cần phân tích dưới hình:

- Các máy ảo được kết nối với nhau qua Tailscale VPN Mesh, giúp liên lạc an toàn xuyên qua các mạng vật lý khác nhau.
- Client truy cập website thông qua NGINX (VM1) tại địa chỉ https://shop-ecommerce.local (mọi truy cập HTTP ở cổng 80 đều được chuyển hướng 301 sang HTTPS ở cổng 443); đây là điểm vào duy nhất của hệ thống.
- NGINX phân phối yêu cầu đến hai web server: Apache (Linux) và IIS (Windows).
- Cả hai web server cùng kết nối đến máy chủ cơ sở dữ liệu VM4 qua cổng 3306.
- VM5 cung cấp dịch vụ FTP để lập trình viên tải mã nguồn lên máy chủ; từ đây mã nguồn được sao chép thủ công sang từng web server.

1.6. Sơ đồ luồng hoạt động hệ thống



Hình 1.2. Sơ đồ luồng hoạt động: mua sắm và cập nhật mã nguồn

Luồng mua sắm: Khách hàng gửi yêu cầu đến NGINX; NGINX chọn một backend (Apache hoặc IIS) theo thuật toán Round-Robin; backend truy vấn hoặc ghi dữ liệu vào cơ sở dữ liệu VM4; kết quả được trả về dưới dạng JSON và hiển thị cho khách hàng.

Luồng cập nhật mã nguồn: Lập trình viên kết nối VPN, tải mã nguồn lên FTP Server (VM5); sau đó mã nguồn được sao chép thủ công sang thư mục web của Apache trên VM2 (/var/www/html) và của IIS trên VM3 (C:\inetpub\wwwroot). Hệ thống chưa có cơ chế đồng bộ mã nguồn tự động giữa các web server.

1.7. Phân tích các chức năng chính

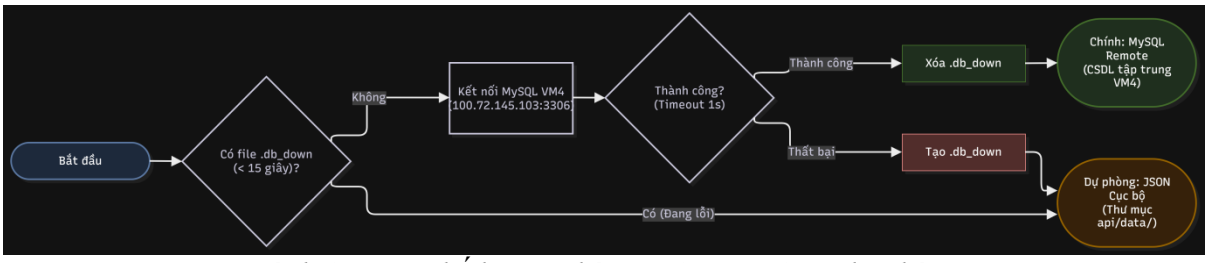
Chức năng	Nội dung
Kết nối mạng riêng	Các máy ảo giao tiếp thông qua Tailscale VPN Mesh.
Reverse proxy và cân bằng tải	NGINX tiếp nhận yêu cầu và phân phối đến hai web server theo Round-Robin.
Hosting đa nền tảng	Apache trên Linux và IIS trên Windows cùng phục vụ một ứng dụng.
Quản lý dữ liệu tập trung	Hai web server kết nối đến MariaDB/MySQL trên VM4 (cơ sở dữ liệu `phone_store`).
Triển khai mã nguồn qua FTP	FTP (vsftpd) nhận file và giới hạn người dùng bằng chroot; mã nguồn sau đó được sao chép thủ công sang từng web server.
Giám sát và kiểm thử tải	GoAccess chạy trên VM1 (AlmaLinux – NGINX Load Balancer), phân tích log truy cập của NGINX và hiển thị Dashboard thời gian thực qua WebSocket cổng 7890 tại đường dẫn /dashboard/; Apache Benchmark (ab) tạo tải kiểm thử.

Chức năng	Nội dung
Chuyển đổi khi có sự cố	Khi một backend ngừng hoạt động, NGINX chuyển yêu cầu sang backend còn lại.

Để phục vụ kiểm thử, ứng dụng cung cấp các RESTful API sau (định dạng JSON):

Endpoint	Phương thức	Chức năng
`/api/products.php`	GET, POST	Lấy danh sách sản phẩm; thêm, sửa, xóa, cập nhật kho
`/api/orders.php`	GET, POST	Xem đơn hàng; tạo đơn mới, đổi trạng thái
`/api/users.php`	GET, POST	Danh sách và đăng ký tài khoản khách hàng
`/api/dashboard.php`	GET	Thống kê doanh thu, tổng đơn, tồn kho cho trang quản trị
`/api/server-info.php`	GET	Trả về thông tin node đang phục vụ (phục vụ kiểm thử cân bằng tải)

Ứng dụng còn có cơ chế lưu trữ kép (Dual-Storage Fallback): lớp kết nối `db.php` kiểm tra kết nối tới VM4:3306; nếu thành công thì đọc/ghi MySQL, nếu thất bại thì chuyển sang đọc/ghi các file JSON cục bộ trong `api/data/`. [Chỉ đưa cơ chế này vào phần "đã đạt được" khi nhóm đã thật sự ngắt DB để kiểm thử.]



Hình 1.3. Cơ chế lưu trữ kép MySQL – JSON dự phòng

CHƯƠNG 2. TRIỂN KHAI KỸ THUẬT

Mỗi mục trong chương này trình bày theo thứ tự: mục đích, môi trường, các bước chính, cách kiểm tra và hình ảnh minh chứng. Các lệnh và file cấu hình đầy đủ được đặt ở Phụ lục.

2.1. Môi trường triển khai

[Nêu công cụ ảo hóa (VirtualBox/VMware/…), máy thật, chế độ mạng của máy ảo và các dịch vụ cần cài đặt.]

Máy ảo	Hệ điều hành	CPU	RAM	Ổ đĩa	Chế độ mạng
VM1	AlmaLinux	[...]	[...]	[...]	[...]
VM2	CentOS	[...]	[...]	[...]	[...]
VM3	[...]	[...]	[...]	[...]	[...]
VM4	Ubuntu	[...]	[...]	[...]	[...]
VM5	[...]	[...]	[...]	[...]	[...]

2.2. Thiết lập Tailscale VPN Mesh

Mục đích. tạo mạng riêng ảo để 5 máy ảo giao tiếp an toàn, mỗi máy nhận một địa chỉ IP dải 100.x.x.x.

Các bước chính. cài Tailscale trên cả 5 máy (Linux: script cài đặt chính thức; Windows Server: file cài đặt `.exe`), chạy `sudo tailscale up` và xác thực thiết bị trên Admin Console; có thể đổi hostname trên Admin Console cho dễ quản lý.

```
curl -fsSL https://tailscale.com/install.sh | sh
sudo tailscale up
```

Kiểm tra. dùng `tailscale status` và `ping` giữa các node để xác nhận kết nối.

[CHÈN HÌNH: Ảnh Admin Console hiển thị 5 máy và địa chỉ IP]

Hình 2.1. Danh sách máy trong Tailscale Admin Console

[CHÈN HÌNH: Ảnh `tailscale status` / ping giữa các VM]

Hình 2.2. Kết quả kiểm tra kết nối giữa các node

2.3. Cấu hình VM1 – NGINX Reverse Proxy và Load Balancer

Mục đích: Đóng vai trò là điểm tiếp nhận duy nhất cho mọi truy cập từ client; thực hiện phân tải lưu lượng đến hai backend (Apache, IIS) và tự động loại bỏ các node lỗi khỏi cụm máy chủ. Đồng thời, cấu hình mã hóa bảo mật (HTTPS) và tích hợp hệ thống giám sát thời gian thực GoAccess.

Các bước chính:

Cài đặt NGINX trên AlmaLinux và tiến hành tạo tệp tin cấu hình chính tại đường dẫn /etc/nginx/conf.d/loadbalancer.conf. Cấu trúc tệp tin được chia thành các khối chức năng như sau:

```
# ========================================
# NGINX LOAD BALANCER
# Windows + Linux Backend qua Tailscale
# ========================================
upstream backend_servers {
    # Node Linux Web Server (CentOS Apache)
    server 100.86.108.58:80 max_fails=2 fail_timeout=5s;

    # Node Windows Web Server (Windows Server IIS)
    server 100.109.69.94:80 max_fails=2 fail_timeout=5s;

    keepalive 32;
}

# ========================================
# HTTP -> HTTPS REDIRECTION (PORT 80)
# ========================================
server {
    listen 80;
    server_name _;

    return 301 https://$host$request_uri;
}

# ========================================
# HTTPS SERVER & LOAD BALANCER (PORT 443)
# ========================================
server {
    listen 443 ssl http2;
    server_name _;

    # SSL Certificates
    ssl_certificate /etc/nginx/ssl/nginx.crt;
    ssl_certificate_key /etc/nginx/ssl/nginx.key;

    # SSL Protocols & Ciphers
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;
    ssl_prefer_server_ciphers on;
    ssl_session_cache shared:SSL:10m;
    ssl_session_timeout 10m;

    # Security Headers: Mixed Content Resolution
    add_header Content-Security-Policy "upgrade-insecure-requests" always;
    add_header X-Content-Type-Options "nosniff" always;
```

```
# Standard access log for GoAccess
access_log /var/log/nginx/access.log combined;

# ====================================
# GOACCESS DASHBOARD
# ====================================
location = /dashboard {
    return 301 /dashboard/;
}

location = /dashboard/ {
    root /usr/share/nginx/html;
    try_files /report.html =404;
    default_type text/html;
    access_log off;
    add_header Cache-Control "no-cache, no-store, must-revalidate";
}

# ====================================
# GOACCESS WEBSOCKET REVERSE PROXY
# ====================================
location /ws {
    proxy_pass http://127.0.0.1:7890;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
    proxy_set_header Host $host;
    proxy_read_timeout 86400s;
    proxy_send_timeout 86400s;
}

# ====================================
# MINIO OBJECT STORAGE PROXY (VM4)
# ====================================
location /storage/ {
    proxy_pass http://100.72.145.103:9000/shop-assets/;
    proxy_http_version 1.1;
    proxy_set_header Host 100.72.145.103:9000;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}

# ====================================
# LOAD BALANCER WEBSITE PROXY
# ====================================
location / {
    proxy_pass http://backend_servers;

    # Keep-Alive Upstream
    proxy_http_version 1.1;
    proxy_set_header Connection "";

    # Client Identity Headers
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;

    # Timeouts & High Availability Fast Failover
    proxy_connect_timeout 2s;
    proxy_send_timeout 10s;
    proxy_read_timeout 10s;
```

```
        proxy_next_upstream error timeout invalid_header http_500 http_502 http_503
http_504;
        proxy_next_upstream_tries 3;

        proxy_buffering on;
    }
}
```

Giải thích kỹ thuật cấu hình:

Cân bằng tải và Dự phòng lỗi (Failover): Khối upstream cấu hình hai máy chủ Backend (CentOS và Windows). Việc không khai báo thuật toán đồng nghĩa với việc NGINX đang sử dụng chiến lược Round-Robin mặc định. Các tham số max_fails=2 và fail_timeout=5s tạo nên cơ chế tự động theo dõi sức khỏe (Health Check) thụ động: nếu một Backend gặp sự cố kết nối liên tiếp 2 lần, nó sẽ bị cô lập trong 5 giây, và chỉ thị proxy_next_upstream sẽ lập tức chuyển hướng lưu lượng sang Backend khỏe mạnh còn lại (proxy_next_upstream_tries 3).

Tối ưu hiệu suất TCP: Chỉ thị keepalive 32 kết hợp cùng proxy_http_version 1.1 và việc làm rỗng Header Connection (proxy_set_header Connection "") thiết lập cơ chế tái sử dụng kết nối TCP giữa Load Balancer và các máy chủ Backend, giúp giảm đáng kể thời gian tải trang. Các Header X-Real-IP, X-Forwarded-For có nhiệm vụ vận chuyển chính xác địa chỉ IP gốc của người dùng xuống tận máy chủ ứng dụng.

Định tuyến tài nguyên tĩnh về MinIO Object Storage: Khối `location /storage/` đóng vai trò Reverse Proxy chuyển tiếp toàn bộ yêu cầu tải ảnh sản phẩm về dịch vụ MinIO đang chạy trên máy chủ VM4 (`100.72.145.103:9000/shop-assets/`), giúp hai máy chủ Backend (Apache và IIS) hiển thị chung nguồn ảnh mà không cần lưu trữ cục bộ.

Bảo mật và Mã hóa: Máy chủ lắng nghe cổng 80 sẽ chặn mọi nỗ lực truy cập không an toàn và chuyển hướng bắt buộc (Mã trạng thái 301) toàn bộ luồng dữ liệu sang giao thức mã hóa HTTPS ở cổng 443 (Sử dụng chứng chỉ tự cấp phát). Để giải quyết triệt để lỗi xung đột nội dung (Mixed Content) làm vỡ giao diện trên trang web, Header Content-Security-Policy "upgrade-insecure-requests" được chèn vào nhằm ép trình duyệt tự động chuyển tất cả các tài nguyên (CSS, JS) chưa an toàn lên HTTPS.

Cấu hình Giám sát thời gian thực với GoAccess: GoAccess được thiết lập chạy nền trực tiếp trên máy chủ VM1 (AlmaLinux). Phần mềm này chịu trách nhiệm phân tích liên tục tệp /var/log/nginx/access.log và khởi tạo kết nối WebSocket cục bộ tại địa chỉ

127.0.0.1:7890. Báo cáo phân tích được phục vụ qua đường dẫn bảo mật /dashboard/. Để khắc phục lỗi kết nối ws:// bị trình duyệt chặn trên môi trường HTTPS, hệ thống sử dụng cấu hình Proxy nội bộ tại location /ws, bọc và định tuyến lại toàn bộ dữ liệu qua cổng 443 an toàn.

Đóng gói kịch bản tự động hóa (start-lab.sh):

Quy trình cấu hình phức tạp này được nhóm đóng gói thành kịch bản Bash Script start-lab.sh. Script đảm bảo quy trình triển khai liền mạch qua các bước: (1) Tự động sao lưu cấu hình NGINX dự phòng; (2) Ghi đè file cấu trúc chuẩn; (3) Khởi tạo và liên kết Systemd Service cho tiến trình GoAccess, đồng bộ hóa cùng tiến trình hệ điều hành; và (4) Thực thi các lệnh curl tự động rà soát trạng thái hoạt động của cổng 80, 443, 7890 cùng bước bắt tay giao thức WebSocket (Handshake) trước khi trả về kết quả sẵn sàng.

Kiểm tra vận hành:

Thay vì thực thi các lệnh thủ công, quy trình kiểm tra được thực hiện toàn diện thông qua kịch bản tự động hóa start-lab.sh. Khi kích hoạt, kịch bản sẽ tuần tự rà soát trạng thái mạng Tailscale, xác thực tính hợp lệ của cấu trúc NGINX (nginx -t), kiểm tra luồng kết nối HTTP đến các máy chủ Backend và khởi tạo tài nguyên cho GoAccess. Khi chu trình hoàn tất với thông báo "HE THONG DA SAN SANG", quản trị viên tiến hành truy cập vào các đường dẫn định tuyến được kịch bản trả về (bao gồm địa chỉ Website và Dashboard). Tại trình duyệt, giao diện trang web được xác nhận tải thành công, đồng thời biểu tượng WebSocket trên trang giám sát GoAccess hiển thị trạng thái xanh (CONNECTED), minh chứng hệ thống Load Balancer đã đi vào hoạt động ổn định.

```
[root@localhost ~]# start-lab.sh
========================================
 KHOI DONG LOAD BALANCER LAB
========================================
[1/4] Kiem tra Tailscale...
Tailscale OK
[2/4] Kiem tra NGINX...
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
NGINX OK
[3/4] Kiem tra Backend...
Backend 100.86.108.58: OK
Backend 100.109.69.94: OK
[4/4] Tao GoAccess Dashboard...
   ______      ___
  / ____/___  /   | _____________  _____
 / / __/ __ \/ /| |/ ___/ ___/ _ \/ ___/ ___/
/ /_/ / /_/ / ___ / /__/ /__/  __(__  |__  )
\____/\____/_/  |_\___/\___/\___/____/____/

The MIT License (MIT) - Copyright (c) 2009-2026 Gerardo Orellana <hello@goaccess.io>
```

```
Cleaning up resources...
GoAccess Dashboard: OK
========================================
 HE THONG DA SAN SANG
========================================
 Web: https://100.73.121.85/
 Dashboard: https://100.73.121.85/dashboard/
========================================
```

Hình 2.3. Kết quả thực thi script start-lab.sh tự động kiểm tra cấu hình NGINX, kết nối Backend và GoAccess.

2.4. Cấu hình VM2 – Linux Apache Web Server

Mục đích. cung cấp dịch vụ web (Backend 1) chạy ứng dụng Phone Store.

Các bước chính. cài Apache và PHP (`php-mysql`), bật `mod_rewrite` và `headers`; sao chép mã nguồn vào `/var/www/html`; đặt `currentNode` trong `config.js` thành "[Server 1: Linux Apache]"; phân quyền thư mục (ghi được vào `api/data`); mở các cổng cần thiết trên tường lửa.

Kiểm tra. truy cập trực tiếp IP của VM2 trên trình duyệt và gọi `/api/server-info.php` để xác nhận web server phản hồi độc lập.

[CHÈN HÌNH: Ảnh trình duyệt truy cập 100.86.108.58]

Hình 2.4. Website Phone Store phục vụ trực tiếp từ Apache (VM2)

2.5. Cấu hình VM3 – Windows IIS Web Server

Mục đích. cung cấp dịch vụ web trên nền tảng Windows (Backend 2), chạy cùng mã nguồn với Apache.

Các bước chính. cài vai trò Web Server (IIS) và CGI trong Server Manager; cài URL Rewrite Module; cài PHP và cấu hình FastCGI cho đuôi `.php`, bật extension `pdo_mysql`; sao chép mã nguồn vào `C:\inetpub\wwwroot`; đặt `currentNode` thành "[Server 2: Windows IIS]"; cấp quyền Write cho `IIS_IUSRS` trên thư mục `api\data`; mở cổng 80 trên Windows Firewall. Tệp `web.config` đảm nhiệm vai trò tương đương `.htaccess` bên Apache.

Tùy chọn. dùng `tailscale funnel 80` để công khai node IIS ra Internet. [Chỉ ghi nếu nhóm có dùng; nêu rõ rủi ro bảo mật khi mở ra Internet.]

[CHÈN HÌNH: Ảnh trình duyệt truy cập 100.109.69.94 và FastCGI/URL Rewrite trong IIS Manager]

Hình 2.5. IIS phản hồi độc lập (VM3)

2.6. Cấu hình VM4 – Database & Object Storage Server

Mục đích: Lưu trữ tập trung dữ liệu sản phẩm, đơn hàng, người dùng (MariaDB/MySQL) và lưu trữ tập trung tệp hình ảnh sản phẩm bằng dịch vụ MinIO Object Storage để cả hai web server Apache (VM2) và IIS (VM3) cùng truy xuất.

Các bước chính:

Cấu hình MariaDB/MySQL: Cài đặt MariaDB/MySQL; đặt bind-address = 0.0.0.0 để cho phép kết nối từ xa; dùng UFW giới hạn cổng 3306 chỉ cho dải Tailscale; nạp file database.sql để tạo cơ sở dữ liệu, các bảng products, orders, users và tài khoản kết nối từ xa cho dải 100.%.

Bash
```
# /etc/mysql/mariadb.conf.d/50-server.cnf
bind-address = 0.0.0.0

sudo ufw allow from 100.0.0.0/8 to any port 3306
sudo ufw enable
sudo mysql < database.sql
```

Cấu hình MinIO Object Storage: Cài đặt MinIO Server trên VM4 chạy ở cổng **9000** (API) và cổng **9001** (Web Console); mở tường lửa UFW cho cổng 9000 và 9001. Khởi tạo bucket shop-assets để chứa ảnh sản phẩm và sử dụng công cụ minio-client (mc) hoặc giao diện Console để cấp quyền **Public Read (Anonymous Download)** cho bucket shop-assets, giúp NGINX tải ảnh tĩnh qua đường dẫn /storage/... thành công.

Bash
```
sudo ufw allow from 100.0.0.0/8 to any port 9000
sudo ufw allow from 100.0.0.0/8 to any port 9001
mc alias set local http://127.0.0.1:9000 <USER> <PASS>
mc mb local/shop-assets
mc anonymous set download local/shop-assets
```

Kiểm tra: Từ VM2 và VM3, kết nối tới 100.72.145.103:3306 bằng tài khoản dịch vụ và truy vấn các bảng; đồng thời đăng nhập giao diện quản trị MinIO Console (http://100.72.145.103:9001) kiểm tra bucket shop-assets.

[CHÈN HÌNH: Ảnh SHOW TABLES; trên VM4]

Hình 2.6. Cơ sở dữ liệu và các bảng trên VM4

[CHÈN HÌNH: Ảnh kết nối từ VM2/VM3 tới 100.72.145.103:3306]

Hình 2.7. Kết nối từ web server tới DB

[CHÈN HÌNH: Ảnh giao diện MinIO Console cổng 9001 hiển thị bucket shop-assets]

Hình 2.7b. Giao diện quản trị MinIO Console và bucket shop-assets trên VM4

2.7. Cấu hình VM5 – FTP Server

Mục đích. tách dịch vụ lưu trữ mã nguồn ra khỏi web server để tăng tính bảo mật; lập trình viên chỉ được truy cập đúng thư mục của mình.

FTP. dùng vsftpd, tắt truy cập ẩn danh, bật `chroot_local_user` để khóa người dùng vào thư mục `/var/www/shared_code`, cấu hình dải cổng passive 40000–40100.

```
listen=YES
anonymous_enable=NO
local_enable=YES
write_enable=YES
chroot_local_user=YES
pasv_enable=YES
pasv_min_port=40000
pasv_max_port=40100
```

[CHÈN HÌNH: Ảnh /etc/vsftpd.conf và danh sách user]

Hình 2.8. Cấu hình vsftpd và tài khoản FTP

2.8. Cấu hình phân giải tên miền nội bộ

Tên miền `shop-ecommerce.local` được ánh xạ tới địa chỉ NGINX bằng file `hosts` trên máy client (Windows: `C:\Windows\System32\drivers\etc\hosts`; Linux: `/etc/hosts`). Đây là cấu hình phân giải tên miền nội bộ qua file hosts, không phải một DNS Server riêng.

```
100.73.121.85   shop-ecommerce.local
```

[CHÈN HÌNH: Ảnh trình duyệt mở https://shop-ecommerce.local]

Hình 2.9. Truy cập website bằng tên miền nội bộ

2.9. Tích hợp hệ thống và quy trình kiểm thử

Sau khi cấu hình từng máy, nhóm tích hợp theo thứ tự: (1) xác nhận kết nối Tailscale; (2) xác nhận từng web server chạy độc lập; (3) xác nhận kết nối từ web server tới DB; (4) bật NGINX và kiểm tra cân bằng tải; (5) kiểm thử chịu lỗi, FTP và tải. Kết quả chi tiết được trình bày ở Chương 3. [Điều chỉnh thứ tự cho đúng với thực tế của nhóm.]

CHƯƠNG 3. KẾT QUẢ VẬN HÀNH

Chương 2 giải thích cách cấu hình; chương này chứng minh cấu hình đó hoạt động. Mỗi bài kiểm thử gồm mục đích, cách thực hiện, kết quả mong đợi, kết quả thực tế và ảnh minh chứng.

[Toàn bộ mục "Kết quả thực tế" bên dưới phải được điền từ kết quả nhóm đo được. Không dùng số liệu mẫu trong tài liệu dự án.]

3.1. Kết quả triển khai tổng thể

[Tóm tắt các thành phần đã chạy được: Tailscale, NGINX, Apache, IIS, DB, FTP, GoAccess.]

[CHÈN HÌNH: Ảnh tổng hợp (ví dụ: Tailscale Admin + website + GoAccess)]

Hình 3.1. Toàn cảnh hệ thống đang vận hành

3.2. Kiểm thử truy cập website

Mục đích. xác nhận người dùng truy cập được Phone Store qua tên miền nội bộ hoặc IP của NGINX.

Cách thực hiện. mở `https://shop-ecommerce.local` (hoặc `https://100.73.121.85/`) trên máy client; thử xem sản phẩm, đăng ký, đặt hàng và vào trang `/admin`.

Kết quả mong đợi. website hiển thị đầy đủ, các chức năng hoạt động.

Kết quả thực tế. [Điền kết quả thực tế]

[CHÈN HÌNH: Ảnh trình duyệt]

Hình 3.2. Website Phone Store truy cập qua NGINX

3.3. Kiểm thử cân bằng tải

Mục đích. kiểm tra NGINX phân phối yêu cầu luân phiên giữa Apache và IIS.

Cách thực hiện. gửi nhiều yêu cầu đến NGINX (F5 hoặc `curl -sk https://100.73.121.85/api/server-info.php`) và quan sát banner node; đối chiếu với log của từng backend.

Kết quả mong đợi. các yêu cầu được phục vụ luân phiên bởi hai node.

Kết quả thực tế. [Điền kết quả thực tế]

[CHÈN HÌNH: Ảnh 2–3 lần truy cập liên tiếp hoặc kết quả curl]

Hình 3.3. Banner node thay đổi giữa Apache và IIS

3.4. Kiểm thử khả năng chịu lỗi

Mục đích. kiểm tra hệ thống tiếp tục phục vụ khi một backend ngừng hoạt động.

Cách thực hiện. dừng Apache trên VM2 (`sudo systemctl stop httpd`), truy cập liên tục và quan sát; sau đó bật lại Apache và kiểm tra việc phục hồi cân bằng tải.

Kết quả mong đợi. yêu cầu được chuyển sang IIS, người dùng không nhận lỗi 502/504 kéo dài.

Kết quả thực tế. [Điền kết quả thực tế]

Bảng ghi nhận thời gian chuyển đổi (đo bằng vòng lặp `curl -w "%{http_code} %{time_total}"` hoặc công cụ tương đương):

Lần thử	Số request lỗi	Thời gian phản hồi lớn nhất	Ghi chú
1	[...]	[...]	[...]
2	[...]	[...]	[...]
3	[...]	[...]	[...]

[Chỉ nêu mốc "chuyển đổi trong 2 giây" nếu số đo thực tế chứng minh được; cấu hình timeout chưa phải kết quả đo.]

[CHÈN HÌNH: Ảnh/log chứng minh IIS xử lý yêu cầu]

Hình 3.4. Kết quả khi dừng Apache

3.5. Kiểm thử cơ sở dữ liệu và FTP

a) Kết nối cơ sở dữ liệu từ xa.

Mục đích. xác nhận dữ liệu từ ứng dụng được ghi và đọc qua DB tập trung VM4.

Cách thực hiện. tạo đơn hàng hoặc đăng ký tài khoản trên website, sau đó truy vấn trực tiếp bảng `orders`/`users` trên VM4.

Kết quả mong đợi. bản ghi xuất hiện trong DB và hiển thị ở trang quản trị.

Kết quả thực tế. [Điền kết quả thực tế]

[CHÈN HÌNH: Ảnh truy vấn SELECT trên VM4 + trang Admin]

Hình 3.5. Bản ghi đơn hàng trong cơ sở dữ liệu

b) Cơ chế dự phòng JSON. [Chỉ viết nếu nhóm đã tắt DB để thử: mô tả cách ngắt, hiện tượng quan sát được, dữ liệu được ghi vào đâu.]

c) FTP và cô lập người dùng.

Mục đích. xác nhận tài khoản FTP chỉ truy cập được thư mục được phép.

Cách thực hiện. đăng nhập bằng FileZilla với tài khoản thử nghiệm, tải file lên, rồi thử chuyển sang thư mục hệ thống như `/etc`.

Kết quả mong đợi. tải file thành công trong thư mục được cấp; truy cập thư mục ngoài bị từ chối.

Kết quả thực tế. [Điền kết quả thực tế. Lưu ý xác nhận IP máy chủ FTP thực tế (VM5).]

[CHÈN HÌNH: Ảnh FileZilla]

Hình 3.6. Phiên FTP và kết quả giới hạn thư mục

d) Lưu trữ và phân phối ảnh qua MinIO Object Storage.

Mục đích. xác nhận các tệp ảnh sản phẩm lưu trên MinIO (VM4) được phân phối thành công qua NGINX Reverse Proxy.

Cách thực hiện. kiểm tra tệp ảnh trong bucket `shop-assets` trên MinIO Console (cổng 9001), sau đó truy cập trực tiếp đường dẫn ảnh `https://shop-ecommerce.local/storage/...` trên trình duyệt (hoặc `curl -Ik`).

Kết quả mong đợi. máy chủ trả về mã trạng thái `200 OK` và hiển thị đầy đủ hình ảnh sản phẩm trên giao diện web.

Kết quả thực tế. [Điền kết quả thực tế]

[CHÈN HÌNH: Ảnh MinIO Console (cổng 9001) và kết quả truy cập đường dẫn /storage/... trả về 200 OK]

Hình 3.6b. Kết quả truy xuất ảnh sản phẩm từ MinIO qua NGINX

3.6. Kết quả giám sát và kiểm thử tải

Mục đích. đánh giá khả năng chịu tải của cụm và quan sát lưu lượng bằng GoAccess. GoAccess chạy trên VM1 (AlmaLinux – NGINX Load Balancer), hiển thị Dashboard thời gian thực tại đường dẫn /dashboard/ thông qua WebSocket cổng 7890.

Cách thực hiện. từ VM4 chạy ab -n 5000 -c 100 https://100.73.121.85/ (hoặc cấu hình nhóm chọn), đồng thời mở Dashboard GoAccess của VM1 tại https://100.73.121.85/dashboard/ và quan sát trạng thái CONNECTED cùng các chỉ số request, băng thông được cập nhật thời gian thực qua kết nối WebSocket /ws.

Kết quả mong đợi. hệ thống phục vụ hết yêu cầu; biểu đồ GoAccess ghi nhận lưu lượng tăng.

Kết quả thực tế. [Điền kết quả thực tế]

Bảng ghi nhận kết quả Apache Benchmark (điền đúng số liệu công cụ trả về):

Chỉ số	Giá trị đo được
Tổng số yêu cầu (-n) / đồng thời (-c)	[...] / [...]
Complete requests	[...]
Failed requests	[...]
Requests per second	[...]
Time per request (mean)	[...]
Thời gian hoàn thành test	[...]

[CHÈN HÌNH: Ảnh terminal kết quả `ab`]

Hình 3.7. Kết quả Apache Benchmark

[CHÈN HÌNH: Ảnh báo cáo GoAccess]

Hình 3.8. Báo cáo GoAccess trên VM1 trong lúc kiểm thử tải

3.7. Đánh giá kết quả và các vấn đề còn tồn tại

Chức năng	Kết quả kiểm thử	Đánh giá
Truy cập website	[Điền kết quả thực tế]	[Đạt / Chưa đạt]
Cân bằng tải	[Điền kết quả thực tế]	[Đạt / Chưa đạt]

Chức năng	Kết quả kiểm thử	Đánh giá
Chịu lỗi backend	[Điền kết quả thực tế]	[Đạt / Chưa đạt]
Kết nối DB từ xa	[Điền kết quả thực tế]	[Đạt / Chưa đạt]
Dự phòng JSON khi DB lỗi	[Điền kết quả thực tế]	[Đạt / Chưa đạt]
FTP và cô lập người dùng	[Điền kết quả thực tế]	[Đạt / Chưa đạt]
Lưu trữ ảnh MinIO (Object Storage)	[Điền kết quả thực tế]	[Đạt / Chưa đạt]
Giám sát và kiểm thử tải	[Điền kết quả thực tế]	[Đạt / Chưa đạt]

Không bắt buộc mọi hạng mục đều đạt. Với mục chưa hoạt động, hãy ghi đúng tình trạng, nguyên nhân đã xác định và hướng khắc phục.

CHƯƠNG 4. KẾT LUẬN

4.1. Kết quả đạt được

[Tổng hợp những thành phần đã triển khai và kiểm thử thành công: mạng riêng Tailscale, hai web server Apache/IIS, cân bằng tải NGINX, DB tập trung, FTP, GoAccess/ab… Chỉ ghi những gì có minh chứng ở Chương 3.]

4.2. Hạn chế

[Gợi ý các điểm có thể phân tích, chỉ giữ nếu đúng với hệ thống của nhóm:]

- NGINX (VM1) và DB (VM4) đều là điểm đơn (single point of failure); chưa có dự phòng cho hai thành phần này.
- Khi DB lỗi, mỗi web server ghi vào file JSON cục bộ riêng nên dữ liệu giữa Apache và IIS có thể không đồng nhất.
- FTP chưa mã hóa (không dùng FTPS/SFTP) và tùy chọn `allow_writeable_chroot` làm giảm mức cô lập.
- Luật tường lửa cho cổng 3306 dùng dải `100.0.0.0/8`, rộng hơn dải Tailscale `100.64.0.0/10`.
- HTTPS trên NGINX sử dụng chứng chỉ tự ký (self-signed) nên trình duyệt vẫn hiện cảnh báo ở lần truy cập đầu; tên miền nội bộ dùng file hosts thay vì DNS Server.
- Kết quả kiểm thử giới hạn trong môi trường máy ảo của nhóm.
- Chưa triển khai được hệ thống đồng bộ mã nguồn tự động, phải copy thủ công lên từng máy.

4.3. Hướng phát triển

- Thay chứng chỉ HTTPS tự ký bằng chứng chỉ tin cậy từ CA (ví dụ Let's Encrypt); dùng FTPS hoặc SFTP cho truyền file thay cho FTP thuần.
- Dự phòng cho NGINX (keepalived) và cơ sở dữ liệu (replication/cluster).
- Hoàn thiện giám sát và cảnh báo; tối ưu thuật toán cân bằng tải (ví dụ `least_conn`, health check chủ động).
- Xây dựng DNS Server nội bộ thay cho file hosts.
- Triển khai cơ chế đồng bộ mã nguồn tự động giữa các web server (ví dụ NFS hoặc pipeline CI/CD) để thay thế việc sao chép thủ công.

[Phân biệt rõ phần đã làm và phần mới chỉ là đề xuất.]

TÀI LIỆU THAM KHẢO

[1] NGINX, “HTTP Load Balancing”, NGINX Documentation. Available: https://docs.nginx.com/nginx/admin-guide/load-balancer/http-load-balancer/

[2] Microsoft, “Internet Information Services (IIS)”, Microsoft Learn. Available: https://learn.microsoft.com/en-us/iis

[3] The Apache Software Foundation, “Apache HTTP Server Project”. Available: https://httpd.apache.org

[4] Tailscale, “Tailscale Knowledge Base”. Available: https://tailscale.com/kb

[5] vsftpd, “vsftpd.conf”. Available: https://security.appspot.com/vsftpd/vsftpd_conf.html

[6] MariaDB Foundation, “MariaDB”. Available: https://mariadb.org

[7] GoAccess, “GoAccess – Visual Web Log Analyzer”. Available: https://goaccess.io

[8] The Apache Software Foundation, “ab – Apache HTTP server benchmarking tool”. Available: https://httpd.apache.org/docs/2.4/programs/ab.html

[9] The PHP Group, “PHP”. Available: https://www.php.net

[10] Meta, “React”. Available: https://react.dev

[11] IETF, “RFC 959 – File Transfer Protocol”. Available: https://datatracker.ietf.org/doc/html/rfc959

[Chỉ giữ các nguồn thực sự đã dùng; thêm ngày truy cập nếu quy định môn học yêu cầu.]

PHỤ LỤC

Phụ lục A. Cấu hình NGINX đầy đủ

```
# ========================================
# NGINX LOAD BALANCER
# Windows + Linux Backend qua Tailscale
# ========================================
upstream backend_servers {
    # Node Linux Web Server (CentOS Apache)
    server 100.86.108.58:80 max_fails=2 fail_timeout=5s;

    # Node Windows Web Server (Windows Server IIS)
    server 100.109.69.94:80 max_fails=2 fail_timeout=5s;

    keepalive 32;
}

# ========================================
# HTTP -> HTTPS REDIRECTION (PORT 80)
# ========================================
server {
    listen 80;
    server_name _;

    return 301 https://$host$request_uri;
}

# ========================================
# HTTPS SERVER & LOAD BALANCER (PORT 443)
# ========================================
server {
    listen 443 ssl http2;
    server_name _;

    # SSL Certificates
    ssl_certificate /etc/nginx/ssl/nginx.crt;
    ssl_certificate_key /etc/nginx/ssl/nginx.key;

    # SSL Protocols & Ciphers
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;
    ssl_prefer_server_ciphers on;
    ssl_session_cache shared:SSL:10m;
    ssl_session_timeout 10m;

    # Security Headers: Mixed Content Resolution
    add_header Content-Security-Policy "upgrade-insecure-requests" always;
    add_header X-Content-Type-Options "nosniff" always;

    # Standard access log for GoAccess
    access_log /var/log/nginx/access.log combined;

    # ====================================
    # GOACCESS DASHBOARD
    # ====================================
    location = /dashboard {
        return 301 /dashboard/;
    }

    location = /dashboard/ {
        root /usr/share/nginx/html;
        try_files /report.html =404;
```

```
        default_type text/html;
        access_log off;
        add_header Cache-Control "no-cache, no-store, must-revalidate";
    }

    # ====================================
    # GOACCESS WEBSOCKET REVERSE PROXY
    # ====================================
    location /ws {
        proxy_pass http://127.0.0.1:7890;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_read_timeout 86400s;
        proxy_send_timeout 86400s;
    }

    # ====================================
    # MINIO OBJECT STORAGE PROXY (VM4)
    # ====================================
    location /storage/ {
        proxy_pass http://100.72.145.103:9000/shop-assets/;
        proxy_http_version 1.1;
        proxy_set_header Host 100.72.145.103:9000;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # ====================================
    # LOAD BALANCER WEBSITE PROXY
    # ====================================
    location / {
        proxy_pass http://backend_servers;

        # Keep-Alive Upstream
        proxy_http_version 1.1;
        proxy_set_header Connection "";

        # Client Identity Headers
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        # Timeouts & High Availability Fast Failover
        proxy_connect_timeout 2s;
        proxy_send_timeout 10s;
        proxy_read_timeout 10s;
        proxy_next_upstream error timeout invalid_header http_500 http_502 http_503
http_504;
        proxy_next_upstream_tries 3;

        proxy_buffering on;
    }
}
```

Phụ lục B. Lệnh kiểm thử

```
# Cân bằng tải (qua HTTPS với chứng chỉ tự ký)
for i in $(seq 1 10); do curl -sk https://100.73.121.85/api/server-info.php; echo;
done

# Chịu lỗi (dừng/bật Apache trên VM2 - CentOS)
```

```
sudo systemctl stop httpd
sudo systemctl start httpd


# Kiểm thử tải (từ VM4 bắn tải vào cổng HTTPS 443 của VM1)
ab -n 5000 -c 100 https://100.73.121.85/
```

Phụ lục C. Lưu ý khi nộp báo cáo

Không đưa mật khẩu quản trị, mật khẩu DB hoặc thông tin xác thực thật vào báo cáo; che thông tin nhạy cảm trong ảnh chụp và ví dụ cấu hình.